

MAP REDUCE (UNIT 2)

05-Aug-2024

Dr. P. Rambabu, M. Tech., Ph.D., F.I.E.

MapReduce Topics

MapReduce:

- Analyzing the data with Unix Tool
- Analyzing the data using Hadoop
- Hadoop streaming
- Hadoop Pipe

HDFS:

- Design of HDFS, Concepts,
- Basic File system Operations, Interfaces, Data Flow.

Hadoop I/O:

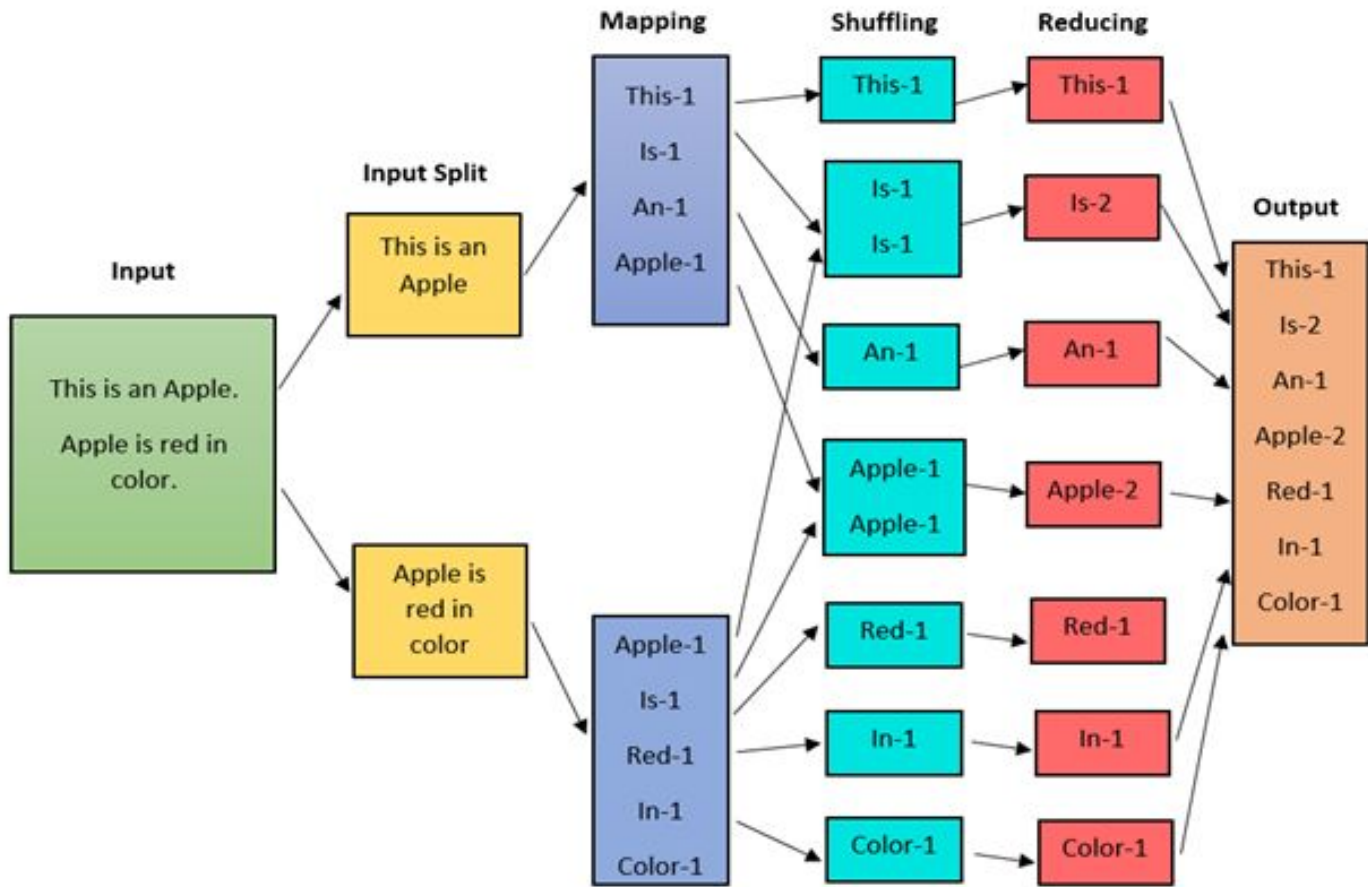
- Data Integrity, Compression, Serialization, File-Based Data Structures

MapReduce

MapReduce is a programming model and processing technique for distributed computing, developed by Google. It is designed to process large datasets in parallel across a distributed cluster of computers, making it a key component of the Hadoop ecosystem.

Key Concepts of MapReduce:

1. Map Function
2. Shuffle and Sort
3. Reduce Function
4. Output



Key Components of MapReduce

1. Map Function:

- The input dataset is divided into smaller, independent chunks that can be processed in parallel.
- The Map function takes these chunks and processes them to produce a set of intermediate key-value pairs.

2. Shuffle and Sort:

The intermediate key-value pairs generated by the Map function are then shuffled and sorted by key. This is an automatic step that ensures that all values associated with the same key are grouped together.

Key Components of MapReduce

3. Reduce Function:

The Reduce function takes the sorted key-value pairs and processes them to produce the final output. This could involve aggregating, filtering, or otherwise processing the values associated with each key.

4. Output:

The final output of the Reduce function is written to a distributed file system, typically HDFS (Hadoop Distributed File System).

Advantages of MapReduce:

- **Scalability:** Can handle large datasets by scaling horizontally across multiple servers.
- **Fault Tolerance:** Automatically handles failures by rerunning tasks on different nodes.
- **Simplicity:** Abstracts complex parallel processing into simple map and reduce operations.

Common Use Cases:

- **Data Processing:** Processing large datasets for analytics, such as log analysis, text processing, and data transformation.
- **Distributed Sorting:** Efficiently sorting large datasets across a distributed system.
- **Data Aggregation:** Aggregating data from various sources, such as calculating averages, sums, or other statistics.

List of Map Reduce Programs

Basic Programs:

1. **Word Count:** Counts the frequency of each word in a dataset.
2. **Max Temperature:** Finds the maximum temperature for each year from a dataset.
3. **Min Temperature:** Finds the minimum temperature for each year.
4. **Average Calculation:** Calculates the average of a set of integers.
5. **Sum Calculation:** Sums up a list of integers.
6. **Inverted Index:** Creates an index that maps each word to the list of document IDs it appears in.

Sorting and Searching:

1. **Secondary Sort:** Sorts data by two keys, one primary and one secondary.
2. **Top N Records:** Finds the top N records from a dataset based on a specific field.
3. **Search Word:** Finds and counts occurrences of a specific word in a dataset.
4. **Sorting Values:** Sorts the values associated with each key.

Data Analysis:

1. **Log Analysis:** Analyzes server log files to count the number of hits per URL.
2. **URL Hit Count:** Counts the number of hits per URL in web server logs.
3. **Sessionization:** Groups user actions into sessions based on timestamps.
4. **User Visit Count:** Counts the number of visits per user.
5. **Page Rank:** Implements the PageRank algorithm to rank web pages.

Joins:

1. **Reduce Side Join:** Joins two datasets using MapReduce, with the join happening in the Reducer.
2. **Map Side Join:** Joins two datasets in the Mapper, often using distributed cache.
3. **Replicated Join:** Joins a large dataset with a small dataset by replicating the smaller one across all nodes

Aggregation:

1. **Data Deduplication:** Removes duplicate records from a dataset.
2. **Counting Unique Users:** Counts the number of unique users based on user IDs.
3. **Sum of Sales:** Calculates the total sales for each product.
4. **Median Calculation:** Calculates the median of a set of numbers.
5. **Finding Mode:** Identifies the most frequent value in a dataset.

Advanced Programs:

1. **TF-IDF Calculation:** Computes Term Frequency-Inverse Document Frequency for terms in documents.
2. **Matrix Multiplication:** Multiplies two matrices using MapReduce.
3. **K-Means Clustering:** Implements the K-Means clustering algorithm.
4. **Naive Bayes Classification:** Classifies data using the Naive Bayes algorithm.
5. **Logistic Regression:** Implements logistic regression for binary classification.

Data Transformations:

1. **Pivot Data:** Transforms rows into columns (pivoting).
2. **Transpose Data:** Transposes a matrix or dataset.
3. **Data Normalization:** Normalizes values in a dataset to a specific range.
4. **Histogram Creation:** Generates a histogram for a dataset.

Data Integration and ETL:

1. **Data Cleanup:** Cleans up dirty data by removing or correcting erroneous entries.
2. **ETL Pipeline:** Extracts, transforms, and loads data into a target system.
3. **Data Masking:** Masks sensitive data to protect privacy.
4. **Data Sampling:** Samples a subset of data from a larger dataset.

Geospatial Processing:

1. **Spatial Join:** Joins spatial datasets based on location proximity.
2. **Polygon Intersection:** Finds the intersection of two sets of polygons.
3. **Distance Calculation:** Calculates the distance between geographic coordinates.
4. **Heat Map Generation:** Creates a heat map from spatial data.

Hadoop Streaming

Hadoop Streaming is a utility that comes with the Hadoop distribution. It allows you to create and run MapReduce jobs with **any executable or script** as the mapper and/or the reducer. This flexibility makes Hadoop Streaming a powerful tool for users who want to leverage the MapReduce programming model but prefer to write their programs in languages other than Java, such as **Python, Ruby, Perl, or even Bash**.

Key Points:

Flexibility: Hadoop Streaming is language-agnostic, meaning **you can use any language** that can read from stdin and write to stdout.

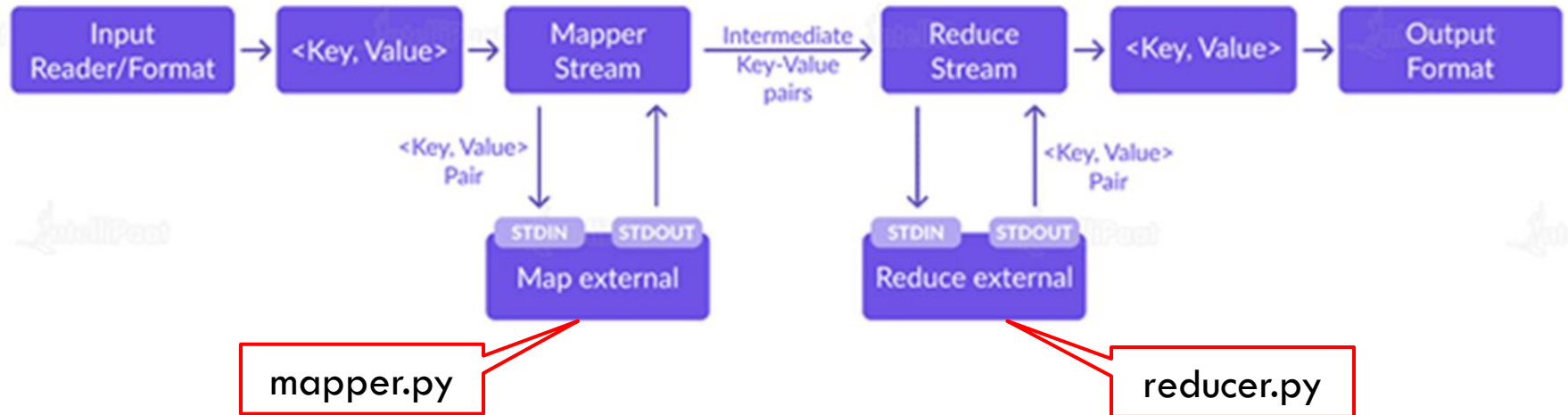
Ease of Use: For those who are more comfortable with scripting languages than with Java, Hadoop Streaming offers an accessible way to leverage Hadoop's distributed processing power.

Use Cases: Common use cases include **log processing, text processing, and data transformation** tasks where writing a full Java MapReduce program would be overkill.

How Hadoop Streaming Works:

- 1.Input Splitting:** Hadoop Streaming reads input files line by line and splits the data into chunks. Each chunk is sent to a mapper instance.
- 2.Mapper Phase:** The mapper reads input line by line from standard input (stdin) and processes it. The output from the mapper is written to standard output (stdout) and is in the form of key-value pairs, typically separated by a tab (\t).
- 3.Shuffling and Sorting:** The Hadoop framework automatically shuffles and sorts the key-value pairs outputted by the mapper. This process ensures that all values associated with a particular key are grouped together before being sent to the reducer.
- 4.Reducer Phase:** The reducer takes the grouped key-value pairs as input, processes them, and writes the output to standard output (stdout). Like the mapper, the reducer can be any executable or script.
- 5.Output:** The output from the reducer is written to the HDFS or local file system

Hadoop Streaming Architecture



WordCount Program

mapper.py

```
import sys
```

```
# Input comes from standard input (stdin)
```

```
for line in sys.stdin:
```

```
    words = line.strip().split()
```

```
    for word in words:
```

```
        print(f"{word}\t1")
```

Running the Hadoop Streaming Job:

```
hadoop jar
```

```
/path/to/hadoop-streaming.jar \
```

```
    -input /path/to/input.txt \
```

```
    -output /path/to/output \
```

```
    -mapper /path/to/mapper.py \
```

```
    -reducer /path/to/reducer.py
```

reducer.py

```
import sys
```

```
current_word = None
```

```
current_count = 0
```

```
# Input comes from standard input (stdin)
```

```
for line in sys.stdin:
```

```
    word, count = line.strip().split('\t')
```

```
    count = int(count)
```

```
    if current_word == word:
```

```
        current_count += count
```

```
    else:
```

```
        if current_word:
```

```
            print(f"{current_word}\t{current_count}")
```

```
        current_word = word
```

```
        current_count = count
```

```
# Don't forget to output the last word
```

```
if current_word == word:
```

```
    print(f"{current_word}\t{current_count}")
```

Design of HDFS

1. **Scalability:** HDFS is built to handle large-scale data sets, often ranging from terabytes to petabytes, distributed across a cluster of machines. This design ensures that the system can easily scale horizontally by adding more machines to the cluster.
2. **Fault Tolerance:** HDFS is designed with fault tolerance in mind. Data is stored in blocks, and each block is replicated across multiple nodes (typically three). This replication ensures that even if some nodes fail, the data remains available.
3. **High Throughput:** HDFS prioritizes high throughput over low latency, making it ideal for applications that require reading and writing large datasets rather than needing quick access to small files.
4. **Write-Once, Read-Many:** HDFS follows a write-once, read-many access model, meaning that files are typically written once and then read multiple times. This model simplifies data coherency and is well-suited for applications like data analysis.
5. **Large Block Size:** HDFS uses large block sizes (128MB or 256MB by default), which reduces the overhead of managing a large number of small files and makes the system more efficient when handling large datasets.

Basic File System Operations

1. Creating a File: When a client wants to create a file, the request is sent to the Namenode, which allocates the blocks and determines which Datanodes will store the data. The client then writes data to the Datanodes, and the Namenode updates the metadata.

2. Reading a File: To read a file, the client first contacts the Namenode to get the file's metadata, including the locations of the data blocks. The client then reads the data directly from the Datanodes.

3. Deleting a File: When a file is deleted, the Namenode marks the file as deleted in its metadata and notifies the Datanodes to remove the corresponding data blocks.

4. Listing Files: Users can list the files and directories in HDFS using commands like `hdfs dfs -ls`. The Namenode provides the metadata needed to display the file structure.

Interfaces

File System Shell: HDFS provides a command-line interface similar to Unix/Linux file system commands, allowing users to perform file operations like copying, moving, and deleting files (`hdfs dfs -put`, `hdfs dfs -get`, etc.).

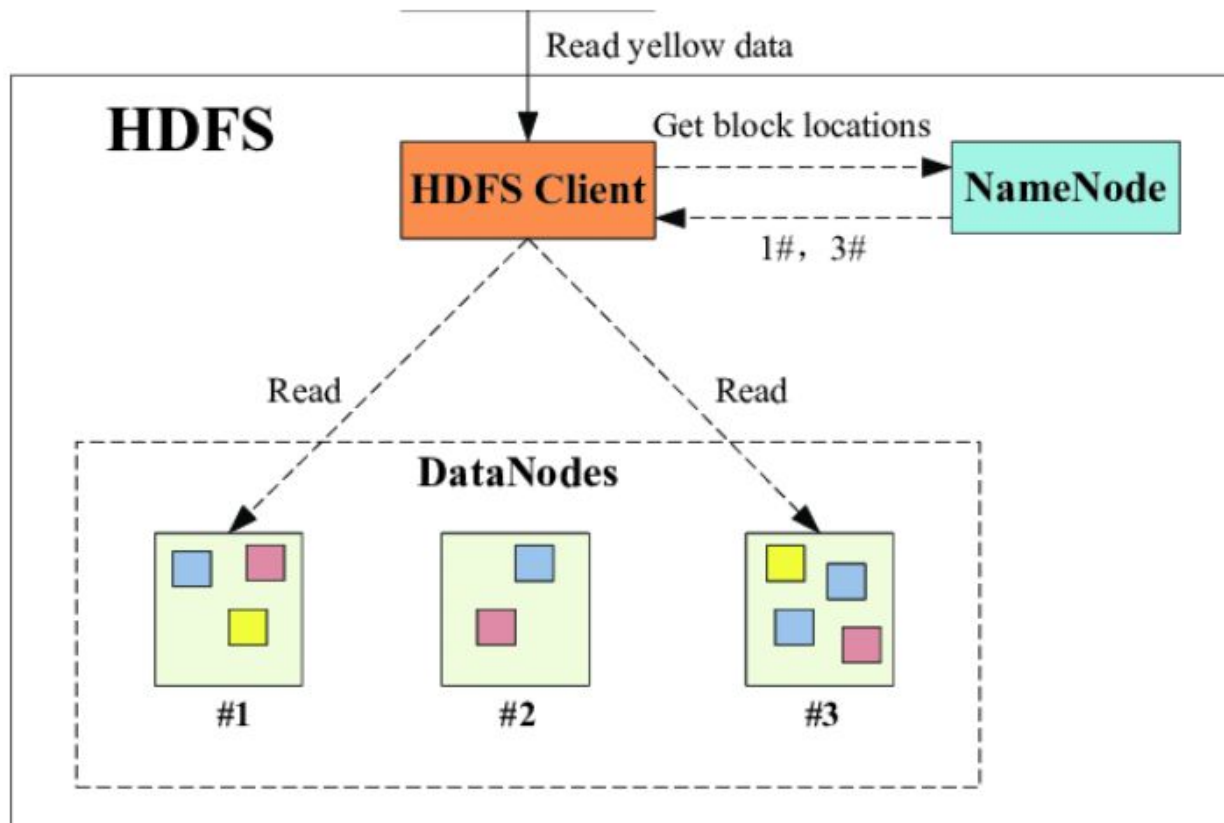
Java API: HDFS offers a **Java API** that developers can use to programmatically interact with the file system, allowing for file operations like **reading, writing, and deleting** files from within Java applications.

REST API: HDFS also provides a RESTful API, which enables applications to interact with HDFS over HTTP, making it accessible from various programming languages and platforms.

Data Flow

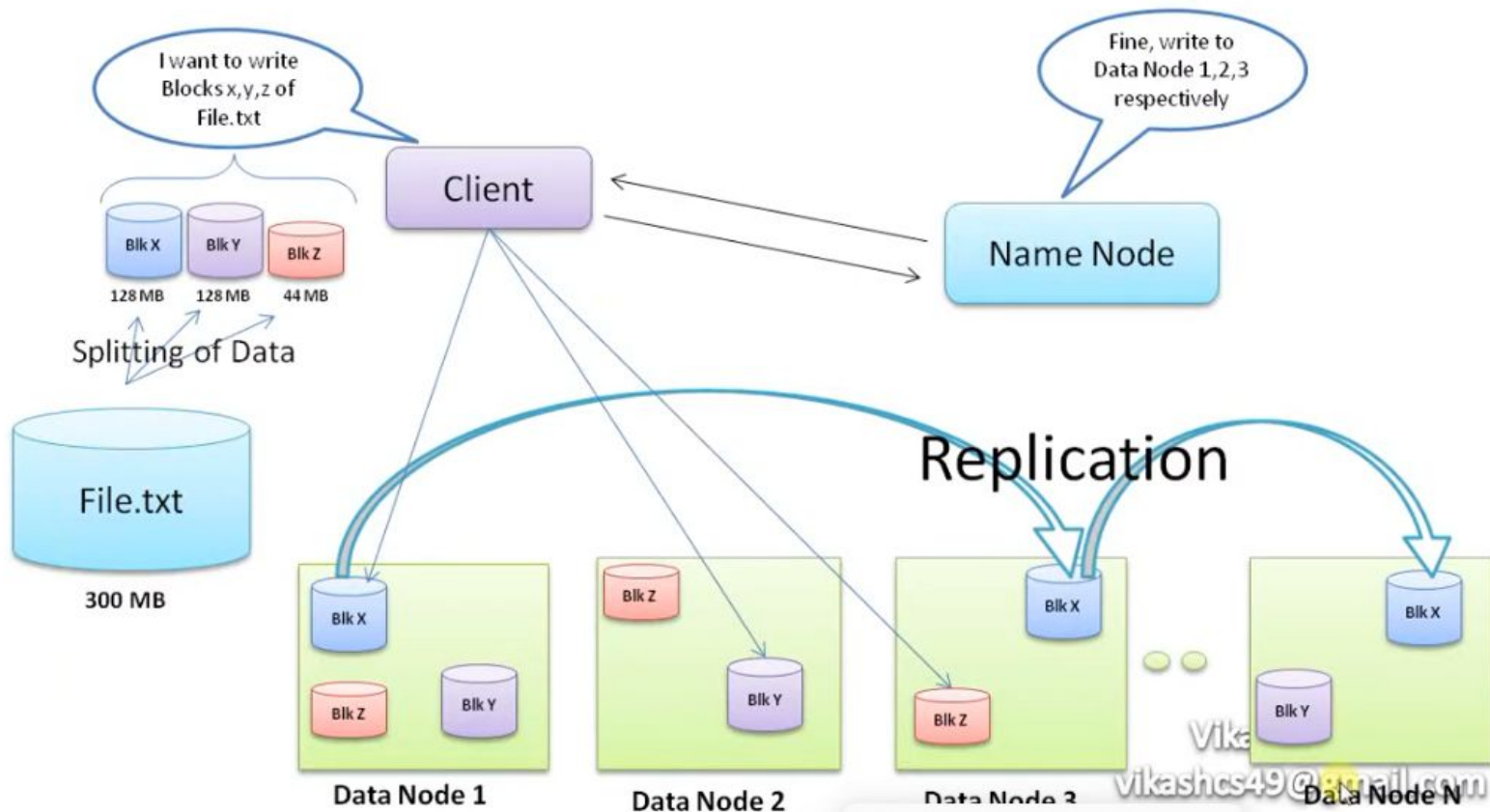
Write Path: When writing data to HDFS, the client first requests permission from the Namenode, which assigns a set of Datanodes to store the data blocks. The client then writes the data to the first Datanode, which replicates it to the other Datanodes as specified by the replication factor. The write operation is considered complete once all replicas are written.

Read Path: For reading data, the client first retrieves the file's metadata from the Namenode, including the list of Datanodes that hold the data blocks. The client then directly reads the data from the Datanodes. If a Datanode is unavailable, the client can retrieve the data from another Datanode holding a replica of the block.



Hadoop Distributed File System (HDFS) structure and its reading process.

Anatomy of File **Write** in HDFS



Hadoop I/O Overview

1. Data Integrity

Purpose: Ensures that data remains accurate and uncorrupted during storage and processing.

Checksums: HDFS uses checksums to verify data integrity. When data is written, a checksum is created and checked during read operations to detect corruption.

Recovery: If data is found to be corrupted, HDFS automatically retrieves a correct replica from another node.

2. Compression

Purpose: Reduces the size of data to save storage space and speed up data transfer across the network.

Common Codecs:

Gzip: High compression but not splittable.

Bzip2: High compression and splittable.

LZO and Snappy: Fast compression and decompression, with lower compression ratios but good for real-time processing.

Application: Used in input/output files, intermediate data in MapReduce, and final outputs to optimize performance.

3. Serialization

Purpose: Converts objects into a byte stream for efficient storage and transmission, then back into objects when needed.

Writable Interface: Hadoop's default framework for serialization, providing efficient handling of basic and custom data types.

Advanced Frameworks: Avro and Protobuf offer more complex serialization with support for schema evolution and richer data structures.

4. File-Based Data Structures

SequenceFile: Stores binary key-value pairs, commonly used for intermediate MapReduce outputs, and supports compression.

Avro: A row-based format that includes a schema, ideal for data interchange and complex data types.

Parquet: A columnar format optimized for read-heavy analytics, allowing efficient access to specific columns.

ORC: Another columnar format, designed for high-performance read operations, especially in Hive, with advanced features like compression and filtering.

Dr. Rambabu Palaka

Professor

School of Engineering

Malla Reddy University, Hyderabad

Mobile: +91-9652665840

Email: drrambabu@mallareddyuniversity.ac.in